CLASSIFICATION

Logistic Regression & Discriminant Analysis

Elsa Ferreira Gomes

2026 / 2027

From regression to classification

A **Linear Regression** model assumes that Y is a **quantitative** variable.

In **Classification**, Y is a **qualitative** (categorical) variable.

Examples of classifiers we'll cover today:

- **Logistic Regression**
- **Discriminant Analysis (LDA / QDA)**

Qualitative variables

Qualitative variables take values in an unordered set C, such as: *eye color* $\in$ {brown, blue, green}.

Given a feature vector X and a qualitative response Y taking values in C, the classification task is:

build a function $C(X)$ that predicts Y from X

Often, we care more about **estimating probabilities** that X belongs to each category, than the classification itself.

Example: it's more valuable to have an estimate of the probability that an insurance claim is fraudulent, than a plain "fraudulent / not fraudulent" label.

MOTIVATING EXAMPLE

Can we use Linear Regression?

The Default dataset (10,000 simulated observations)

Variable	Type	Description
default	Categorical (No / Yes)	Did the customer default on their debt?
student	Categorical (No / Yes)	Is the customer a student?
balance	Numeric	Average credit card balance remaining after payment
income	Numeric	The customer's income

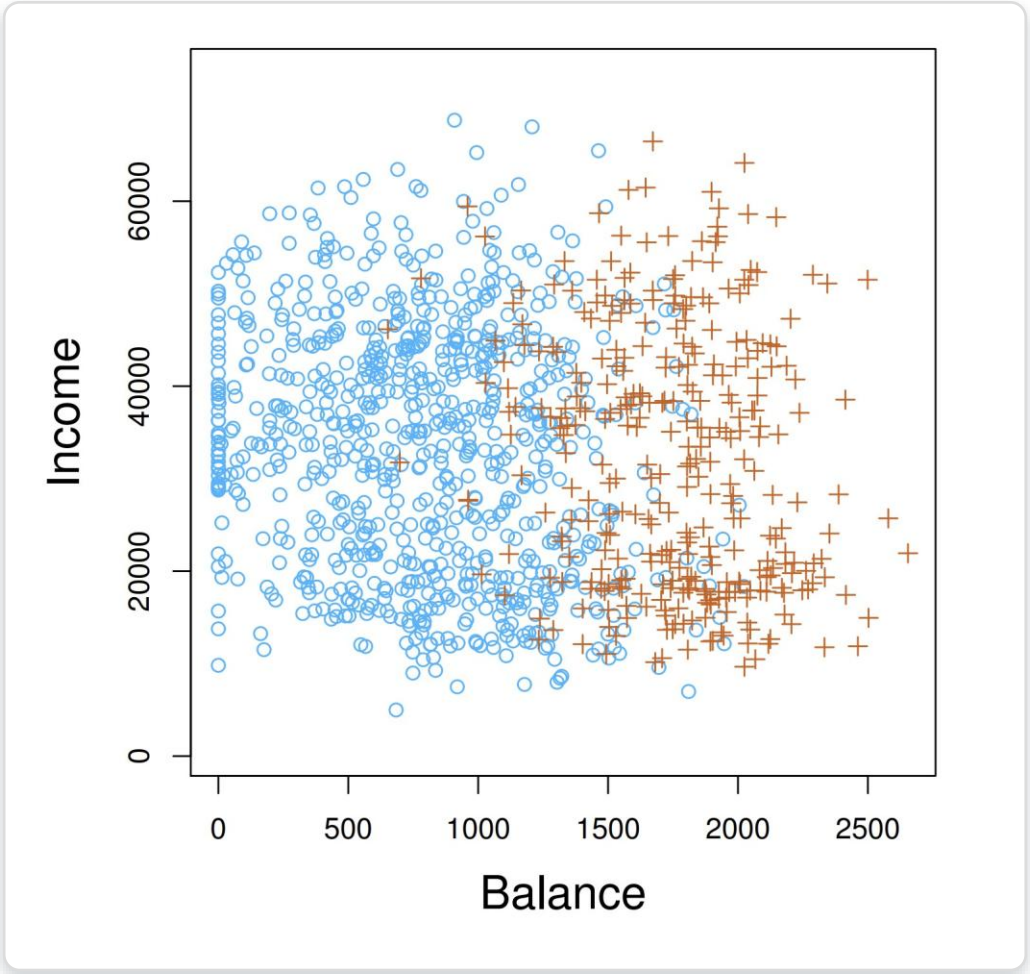
Goal: predict whether a customer will **default**, using balance and income as predictors.

Since default is categorical (not numeric), we first code it as a number:

$$Y = \begin{cases} 0 & \text{if } default = \text{No} \quad \text{(shown in blue)} \\ 1 & \text{if } default = \text{Yes} \quad \text{(shown in brown)} \end{cases}$$

*Can we simply fit a straight line to it, the way we fit **sales** in the previous chapter?*

The Default dataset, visualized



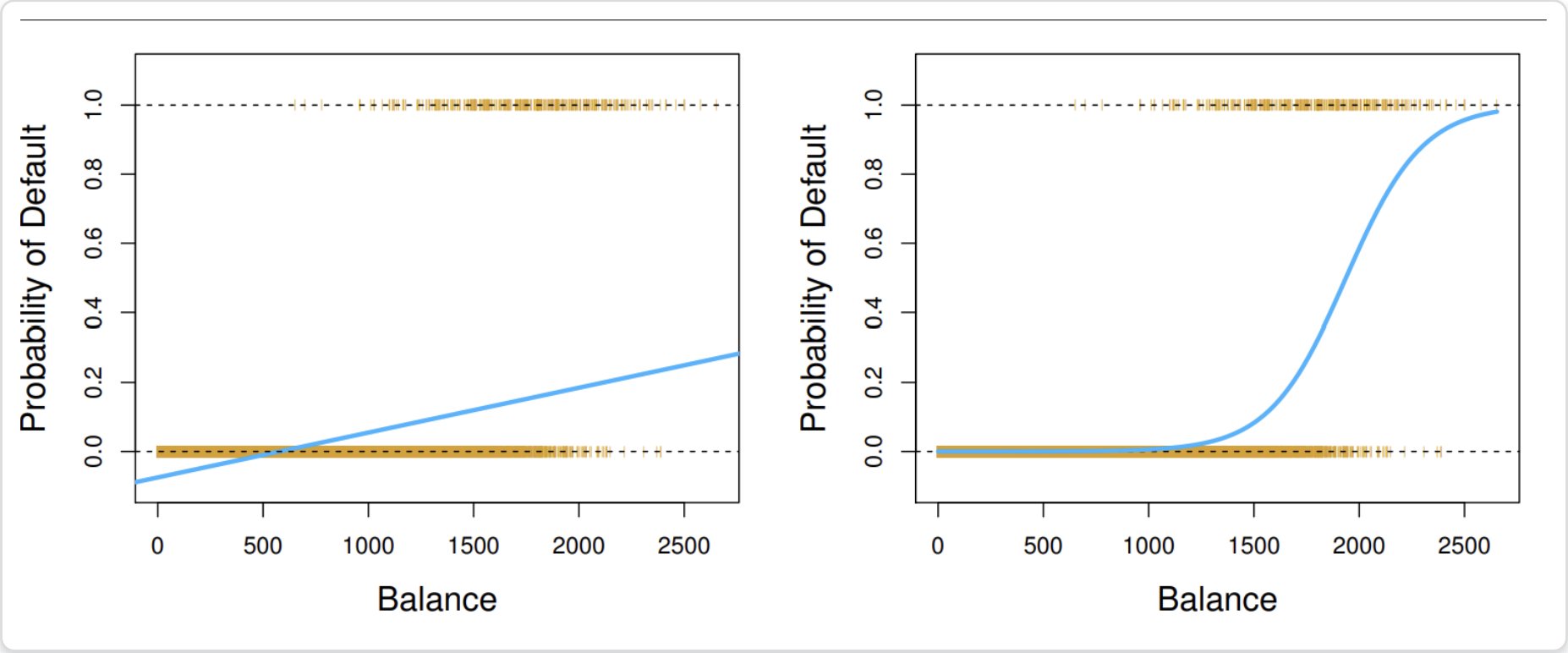
How about a 0.5 threshold?

Idea: perform linear regression of *Y* on *X* and classify as "Yes" if $\hat{y} > 0.5$.

- In a binary case, linear regression does a reasonable job as a classifier, and it's equivalent to Linear Discriminant Analysis (LDA)
- Since, in the population, $E(Y|X=x) = Pr(Y=1|X=x)$, it's tempting to use ordinary regression
- However, linear regression can produce probabilities below 0 or above 1 (which makes no sense as a probability)

Logistic Regression (also called Logit Regression) ensures our estimate for p(X) always lies between 0 and 1.

Linear vs. Logistic: the problem, visualized



Orange marks = indicating the response Y (0 or 1).

Left: Linear Regression, predicted Y can exceed the [0,1] range.

Right: Logistic Regression, predicted Y always lies within [0,1].

Setting up the problem

- Given a set of points X with classes Y (assume binary classification)
- We want a function $f(x)$ that finds the y that maximizes $\Pr(Y=y \mid X=x)$

Defining f: models an output $0 \le p(x) \le 1$:

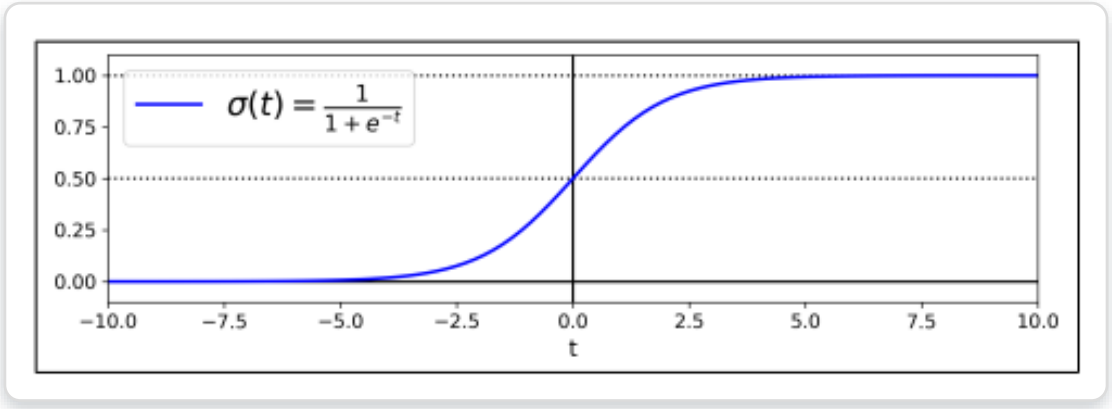
$$f(x) = \text{class}_1 \quad \text{if } p(x) > 0.5$$

$$f(x) = \text{class}_2 \quad \text{if } p(x) \le 0.5$$

To directly approximate the Bayes Classifier: $p(x) \approx \Pr(Y = \text{class}_1 \mid X = x)$

Remember the Logistic function?

$$\sigma(t) = 1 / (1 + e^{-t})$$



σ(t) < 0.5 when t < 0, and **σ(t) ≥ 0.5** when t ≥ 0 — so a Logistic Regression model predicts 1 if X is positive, and 0 if negative.

The logistic function & the logit

Consider $p(x) \approx \Pr(Y = 1 \mid X = x)$:

$$p(x) = \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}} = \frac{1}{1+e^{-(\beta_0+\beta_1 x)}}$$

For any value of β_0, β_1 or X, p(X) is guaranteed to lie in [0, 1].

We can also apply a monotone transformation of p(X) — the logit:

$$\ln\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 x$$

*- p(x) is the **logistic function** (or **sigmoid**);*
- $\beta_0 + \beta_1 x$ is the linear model inside it.

CASE STUDY

Putting it into practice

We now have the logistic function and the logit transform.
Let's apply them to our running example, the **Default** dataset, and actually fit a model.

Loading the Default dataset

PYTHON

```
df = pd.read_csv('Default.csv')

# factorize() returns a label array
# and an array with the unique values
df['default2'] = df.default.factorize()[0]
df['student2'] = df.student.factorize()[0]

df.head(3)
```

default	student	balance	income	default 2	student 2
No	No	729.53	44361.63	0	0
No	Yes	817.18	12106.13	0	1
No	No	1073.55	31767.14	0	0

(*10,000 rows*).

factorize() turns the text categories 'No'/'Yes' into the numeric codes 0/1 that our models need.

Estimating the parameters

We learn (estimate) the parameters β using **Maximum Likelihood Estimation** (MLE).

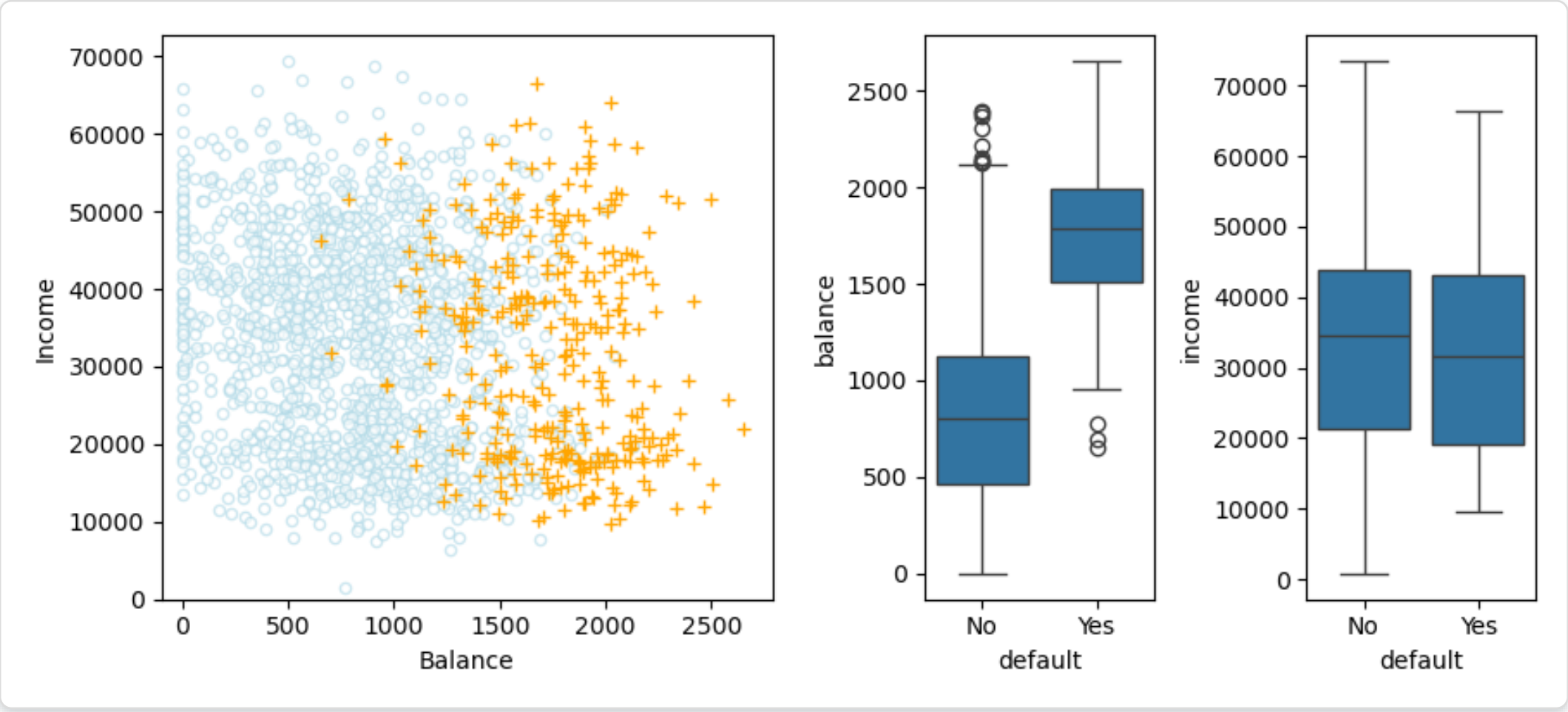
$$l(\beta_0, \beta_1) = \prod_{i:y_i=1} p(x_i) \prod_{i:y_i=0} (1 - p(x_i))$$

(product over i where y_i=1, times product over i where y_i=0)

- Substitute $p(x)$ with the logistic expression
- Take the log-likelihood
- Set the derivative to zero
- Solve: obtain a closed form for the update equations
- Use an iterative solver to find the maximum

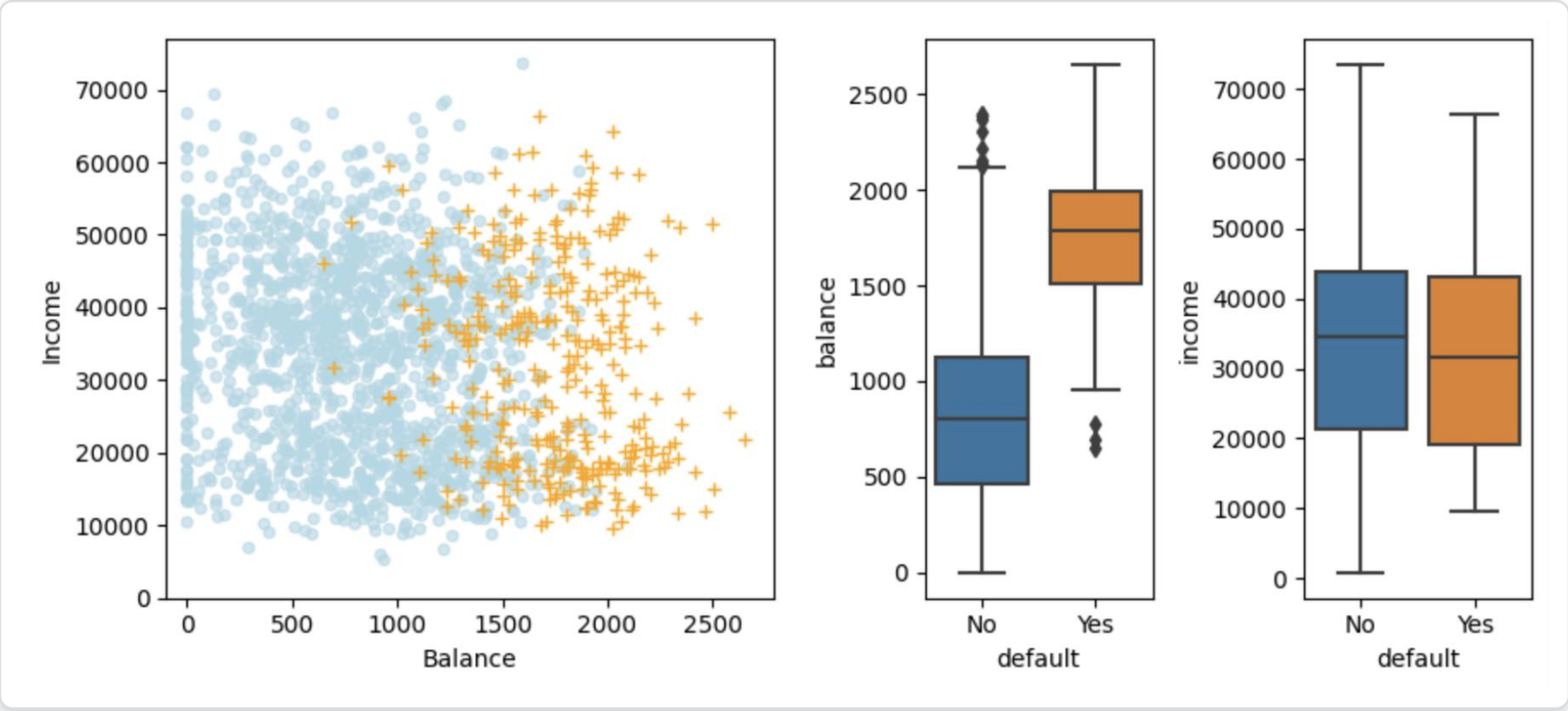
Maximum Likelihood, clearly explained!!!

Visualizing balance and income by default



Orange + marks = customers who defaulted. They clearly cluster at higher balances (income shows far less separation).

Reading the previous chart



Left: income vs. balance, colored by default status.

Center: balance by default status.

Right: income by default status.

Linear vs. Logistic, fit on real data

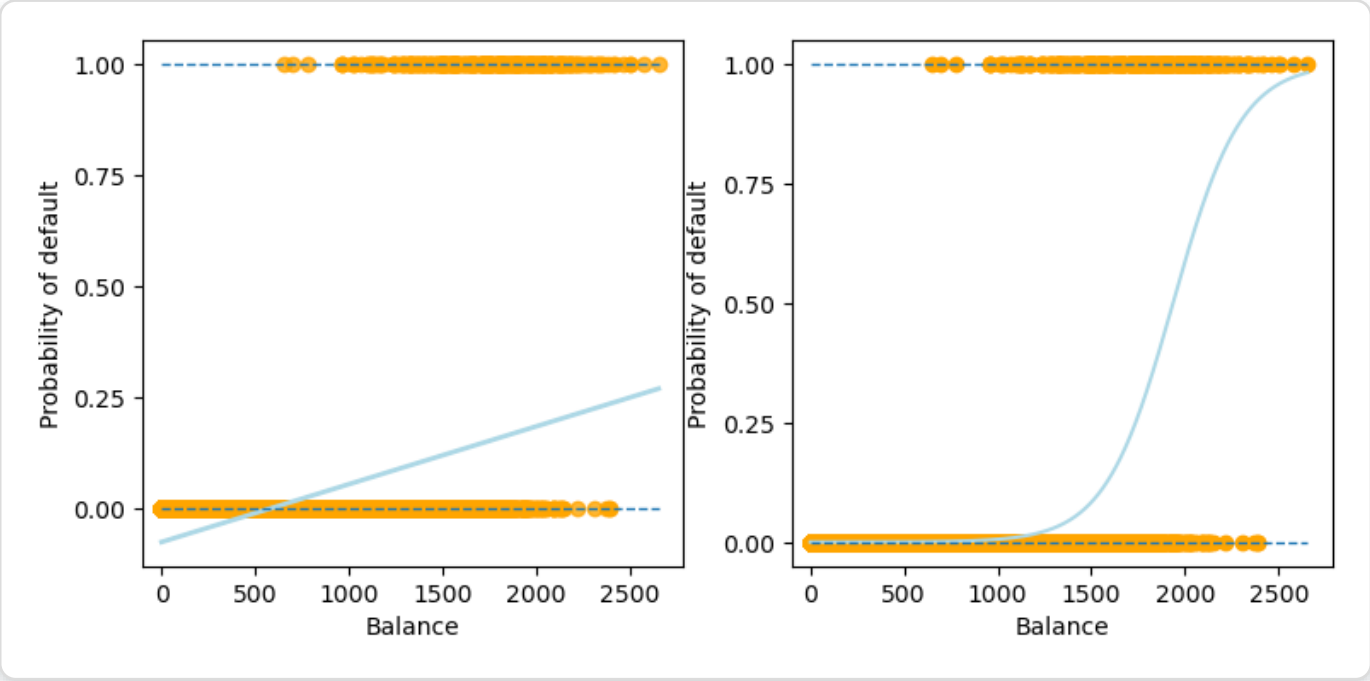
PYTHON

```
clf = skl_lm.LogisticRegression()
clf.fit(X_train, y)
prob = clf.predict_proba(X_test)

# Left: regplot (linear fit)
# Right: logistic curve
```

Notice, we can see predicted probabilities of default using:

- *the linear fit (left) dips below 0 for low balances*
- *the logistic curve (right) never does.*



Fitting: balance predicts default

PYTHON

```
clf = skl_lm.LogisticRegression()

X_train = df.balance.values.reshape(-1,1)
clf.fit(X_train, y)

print('classes:', clf.classes_)
print('coefficients:', clf.coef_)
print('intercept:', clf.intercept_)
```

RESULT

```
classes:      [0, 1]
coefficients: 0.005499
intercept:    -10.6513
```

A positive coefficient means higher balances push the estimated probability of default up.

Making a prediction by hand

What is the estimated probability of default for someone with a balance of $1000?

$$p(1000) = \frac{e^{-10.6513+0.0055\times1000}}{1+e^{-10.6513+0.0055\times1000}} = \frac{1}{1+e^{-(-10.6513+0.0055\times1000)}} = 0.006$$

That's a 0.6% chance of default for a balance of $1000 (reassuringly low, and safely between 0 and 1 thanks to the logistic form).

Statsmodels Logit: default ~ balance

PYTHON

```
X_train = sm.add_constant(df.balance)
est = sm.Logit(y, X_train).fit()
est.summary2().tables[1]
```

	Coef.	Std.Err.	z	P>|z|
const	-10.6513	0.3612	-29.49	< 0.001
balance	0.0055	0.0002	24.95	< 0.001

Both coefficients are highly significant ($p < 0.001$): balance has a real, strong effect on the odds of default.

Statsmodels Logit: default ~ student

PYTHON

```
X_train = sm.add_constant(df.student2)
est = sm.Logit(y, X_train).fit()
est.summary2().tables[1]
```

	Coef.	Std.Err.	z	P>|z|
const	-3.5041	0.0707	-49.55	< 0.001
student[Yes]	0.4049	0.1150	3.52	< 0.001

The positive coefficient says: **being a student is associated with higher odds of default** on its own. Keep this in mind for the next slide.

Multiple Logistic Regression: adding several predictors

$$\ln\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$$

$$p(x) = \frac{e^{\beta_0+\beta_1 x_1+\ldots+\beta_p x_p}}{1+e^{\beta_0+\beta_1 x_1+\ldots+\beta_p x_p}}$$

Example: Confounding

- Students tend to have higher balances than non-students, so their marginal default rate is higher than for non-students
- But for each level of balance, students default less than non-students
- Multiple logistic regression can reveal this. A single-variable model cannot

The confounding, revealed

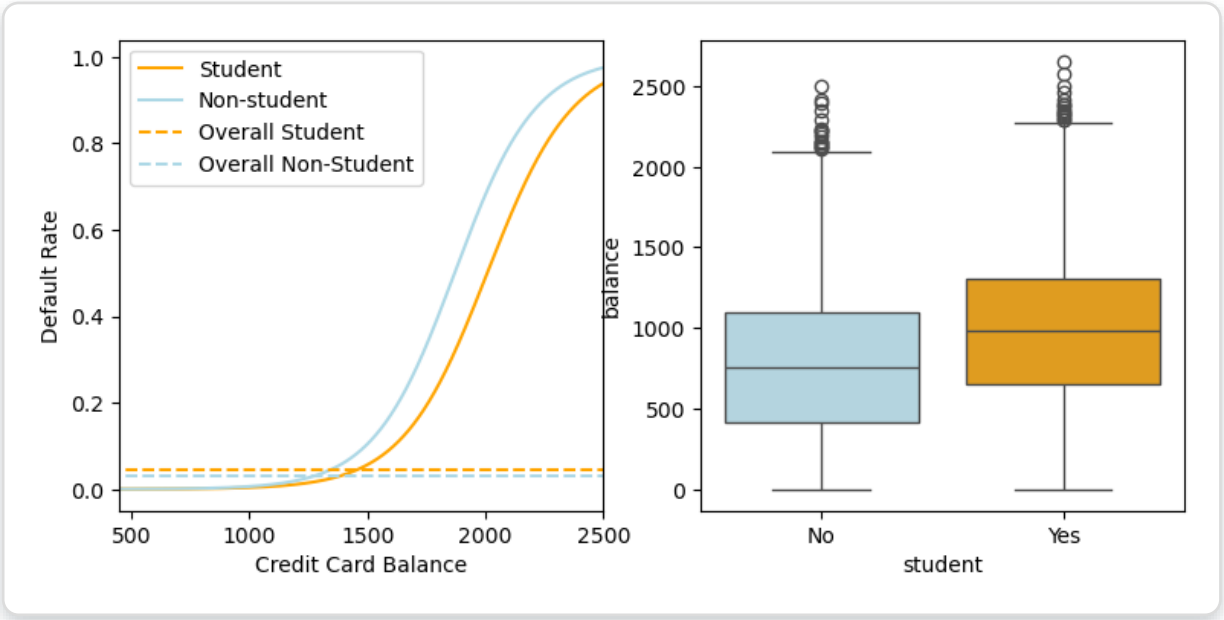
PYTHON

```
X_train = sm.add_constant(
    df[['balance','income','student2']])
est = sm.Logit(y, X_train).fit()
```

	Coef.	Std.Err.	z	P>|z|
const	-10.8690	0.4923	-22.08	< 0.001
balance	0.0057	0.0002	24.74	< 0.001
income	0.0000	0.0000	0.37	0.712
student2	-0.6468	0.2363	-2.74	0.006

The student coefficient flips to **negative** once balance is in the model, confirming students default **less** at the same balance. income is not significant (p = 0.712).

Confounding, visualized



student	default: No	default: Yes
No	6,850	206
Yes	2,817	127

At every fixed balance (orange vs. light-blue curves), students default less but overall (dashed lines), students default more, simply because they tend to carry higher balances.

EXTRA EXAMPLE

A second example: the Iris dataset

Before moving on to Discriminant Analysis, here's a **self-contained illustration** of Logistic Regression — this time on the classic Iris dataset, using a single predictor (petal width) to classify *Iris-Virginica* vs. all other species.

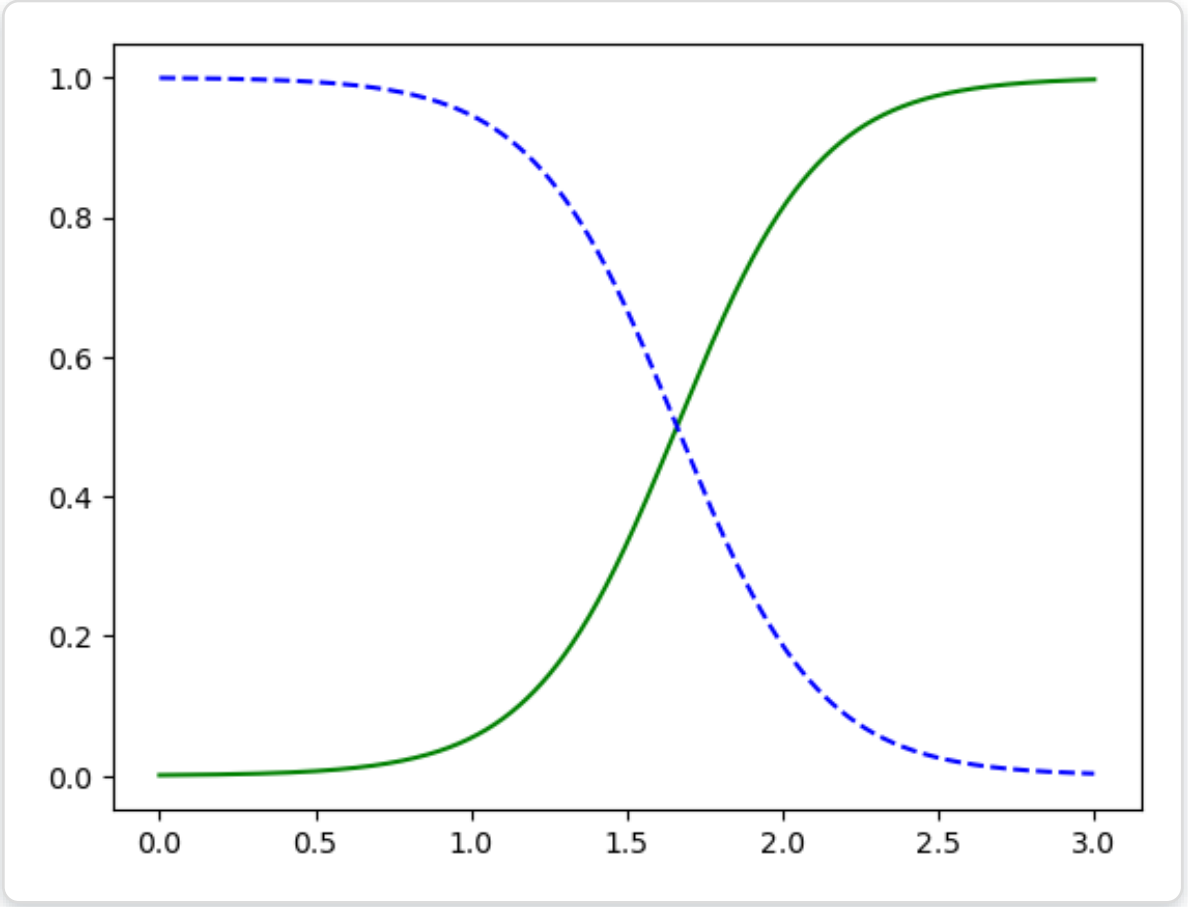
Fitting Logistic Regression on Iris

PYTHON

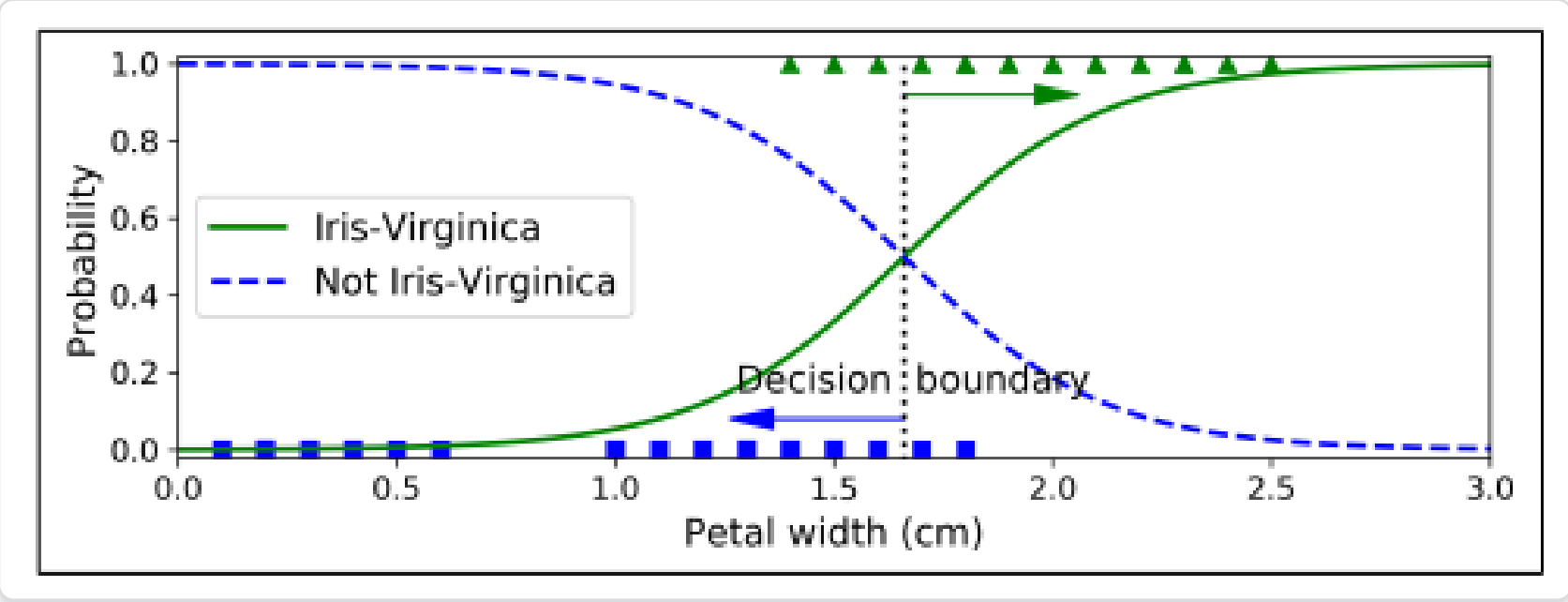
```
iris = datasets.load_iris()
X = iris["data"][:, 3:]  # petal width
y = (iris["target"] == 2).astype(int)
# 1 if Iris-Virginica, else 0

log_reg = LogisticRegression()
log_reg.fit(X, y)

X_new = np.linspace(0, 3, 1000).reshape(-1,1)
y_proba = log_reg.predict_proba(X_new)
```



Estimated probabilities & decision boundary



Where the green (Virginica) and blue (not-Virginica) curves cross is exactly the decision boundary; the petal width at which the model becomes equally confident in either class.

EXTRA EXAMPLE · IRIS

Reading off the decision boundary

PYTHON

```
# the decision boundary is about 1.6
log_reg.predict([[1.7], [1.5]])
```

RESULT

```
array([1, 0])
```

A petal width of 1.7cm → **predicted class 1 (Virginica)**. A petal width of 1.5cm → **predicted class 0 (not Virginica)**. Exactly what we'd expect from a boundary around 1.6cm.

Why Discriminant Analysis?

1. When there is substantial separation between the two classes, Logistic Regression's parameter estimates are surprisingly unstable - Linear Discriminant Analysis does not have this problem.

2. If the predictors X are approximately normally distributed within each class (and the sample size is small), LDA is more stable than Logistic Regression.

3. LDA is popular when there are more than two response classes.

Linear Discriminant Analysis (LDA)

LDA is a machine learning algorithm used to find the linear function that best separates two (or more) classes of data.

The Fisher criterion: choose the decision boundary that maximizes the distance between the means of the two classes, while minimizing the variance within each class.

***The general idea**: model each class's distribution, then flip with Bayes' theorem*

$$\Pr(Y = k|X = x) = \frac{\Pr(X = x|Y = k)\Pr(Y = k)}{\Pr(X = x)}$$

$$\Pr(Y = k|X = x) = \frac{\pi_k f_k(x)}{\sum \pi_l f_l(x)}$$

Using normal (Gaussian) distributions for each class leads to linear or quadratic discriminant analysis.

Estimating the parameters

In practice we don't know the true probabilities. We estimate them from the data:

Parameter	Estimated as
π_k	from the proportion of observations in each class
μ_k	the mean of the points in each class (assuming Gaussian)
σ^2	by averaging the σ^2_k across classes (LDA assumes one shared σ^2)

We don't need to know Pr(X=x) directly, it cancels out. And we're using MLE to estimate μ_k and σ^2.

With more than one predictor in X, we estimate a covariance matrix instead of a single variance.

LDA vs. QDA

L

Linear Discriminant Analysis

Assumes all classes share the same covariance matrix.

→ Results in linear decision boundaries.

Q

Quadratic Discriminant Analysis

Relaxes the equal-covariance assumption: each class gets its own covariance matrix.

→ Results in quadratic (curved) decision boundaries.

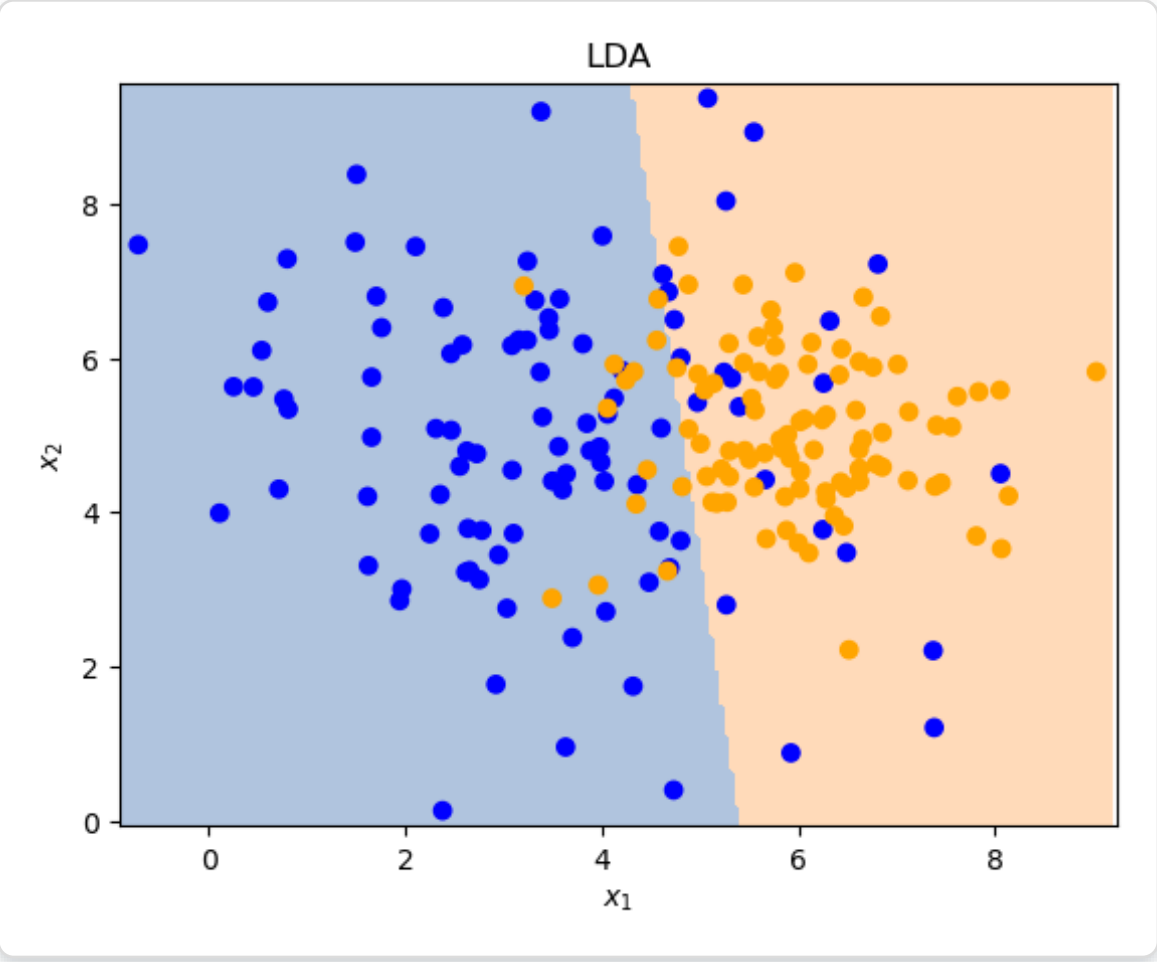
LDA decision boundary, fit live

PYTHON

```
X, y = make_blobs(
    n_samples=[100, 100],
    centers=[(3,5), (6,5)],
    cluster_std=[2, 1])

logr = LogisticRegression().fit(X, y)
plot_classifier_boundary(logr, X)
```

Two synthetic blob classes with different spreads (std 2 vs. std 1) — useful for seeing exactly how the boundary responds to class shape.



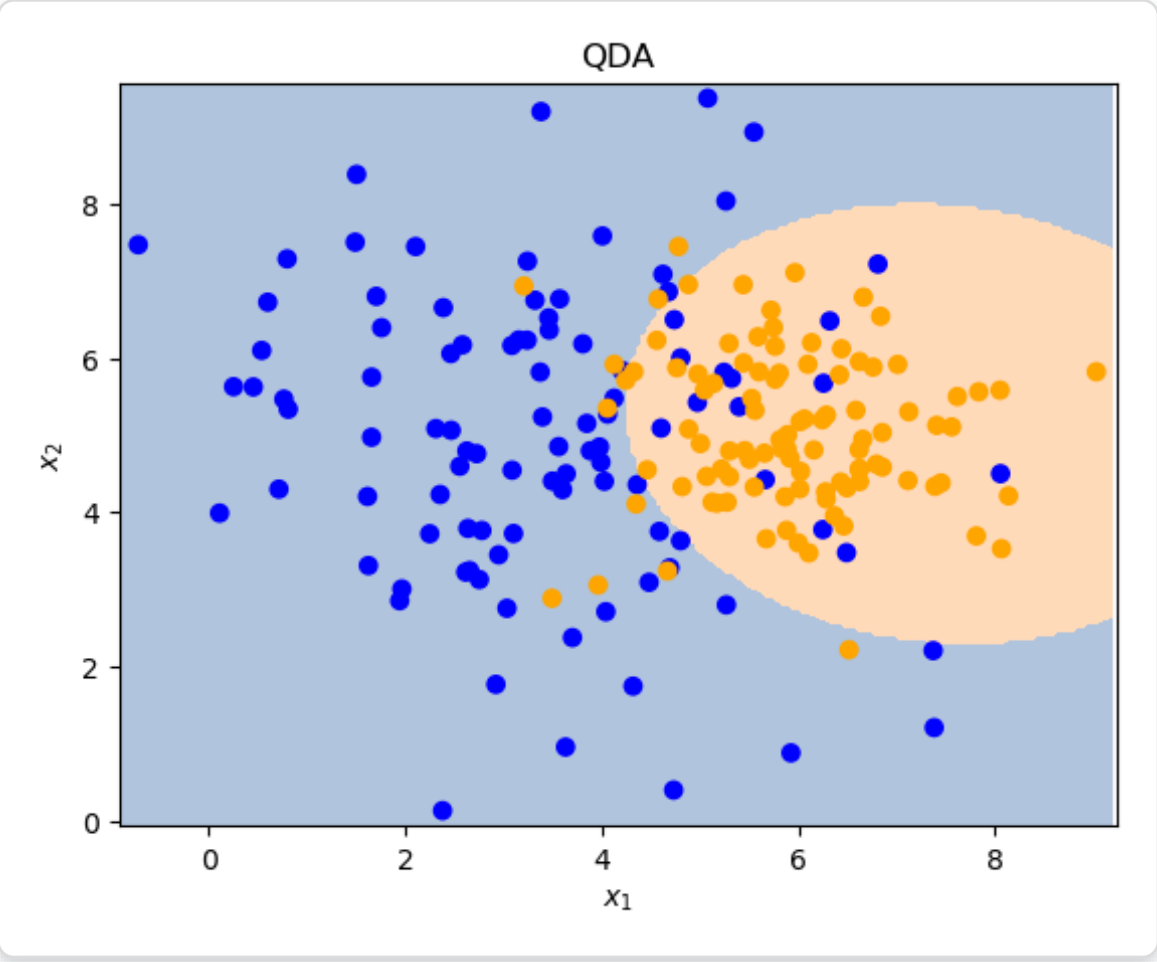
QDA decision boundary, same data

PYTHON

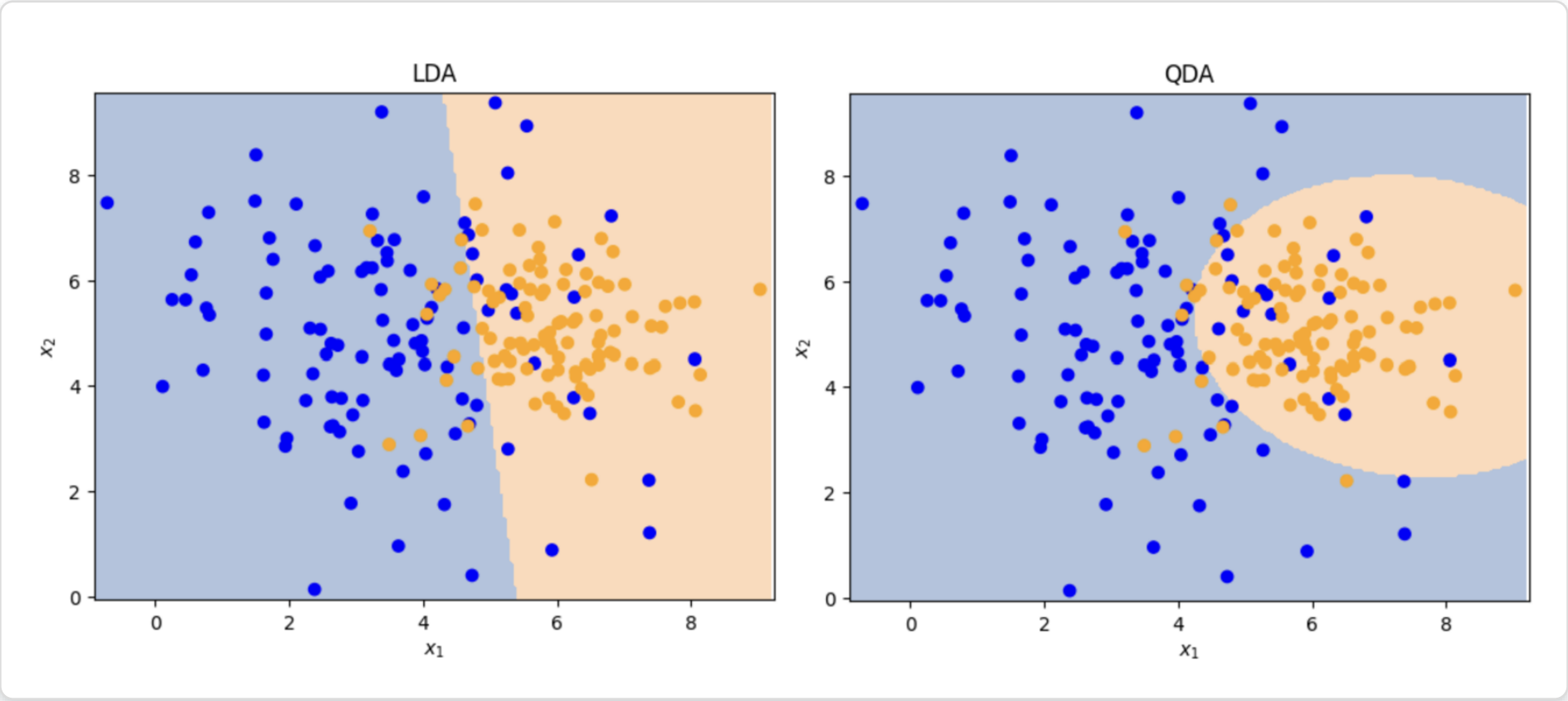
```
discr = QDA().fit(X, y)
plot_classifier_boundary(discr, X)
```

Same two blobs as before. Notice how the boundary curves:

- *QDA can adapt to each class's own spread,*
- *LDA was forced into a straight line.*



LDA vs. QDA, side by side



Confusion matrix — LDA on Default

Compares LDA's predictions to the true default status, for all 10,000 observations (threshold = 0.5):

True default status	No	Yes
Predicted default status		
No	9645	253
Yes	22	80

22 false positives (predicted default, didn't) and **253 false negatives** (predicted no default, but did)
diagonal = correctly classified.

PYTHON

```
lda = LinearDiscriminantAnalysis()
y_pred = lda.fit(X, y).predict(X)
```

Confusion Matrix & Classification Metrics

	Actual: Positive	Actual: Negative
Predicted: Positive	**TP** *True Positive*	**FP** *False Positive*
Predicted: Negative	**FN** *False Negative*	**TN** *True Negative*

Rows = predicted, columns = actual

Accuracy

`(TP+TN) / Total`

overall, how often right

Precision

`TP / (TP+FP)`

of predicted +, how many right

Recall (Sensitivity)

`TP / (TP+FN)`

of real +, how many caught

Specificity

`TN / (TN+FP)`

of real –, how many caught

F1-score

`2×P×R / (P+R)`

harmonic mean of P & R

Precision, recall & f1-score

PYTHON

```
print(classification_report(y, y_pred,
      target_names=['No', 'Yes']))
```

	precision	recall	f1-score	support
No	0.97	1.00	0.99	9667
Yes	0.78	0.24	0.37	333
accuracy			0.97	10000

97% overall accuracy looks great but recall for “Yes” is only 0.24: **the model misses 76% of actual defaulters.**

With only 3.3% of customers defaulting, accuracy alone is misleading.

Lowering the decision threshold

Instead of the default 50% threshold, classify as “Yes” whenever the predicted probability of default exceeds **20%.**

PYTHON

```
decision_prob = 0.2
y_prob = lda.fit(X, y).predict_proba(X)
y_pred_20 = y_prob[:,1] > decision_prob
```

True default status	No	Yes
Predicted default status		
False	9435	140
True	232	193

Yes-recall jumps from 80/333 to 193/333: we now catch far more real defaulters, at the cost of more false alarms (232 vs. 22).

SUMMARY

Logistic Regression vs. LDA

σ

Logistic Regression

Very popular for classification, especially when K = 2 (two classes).

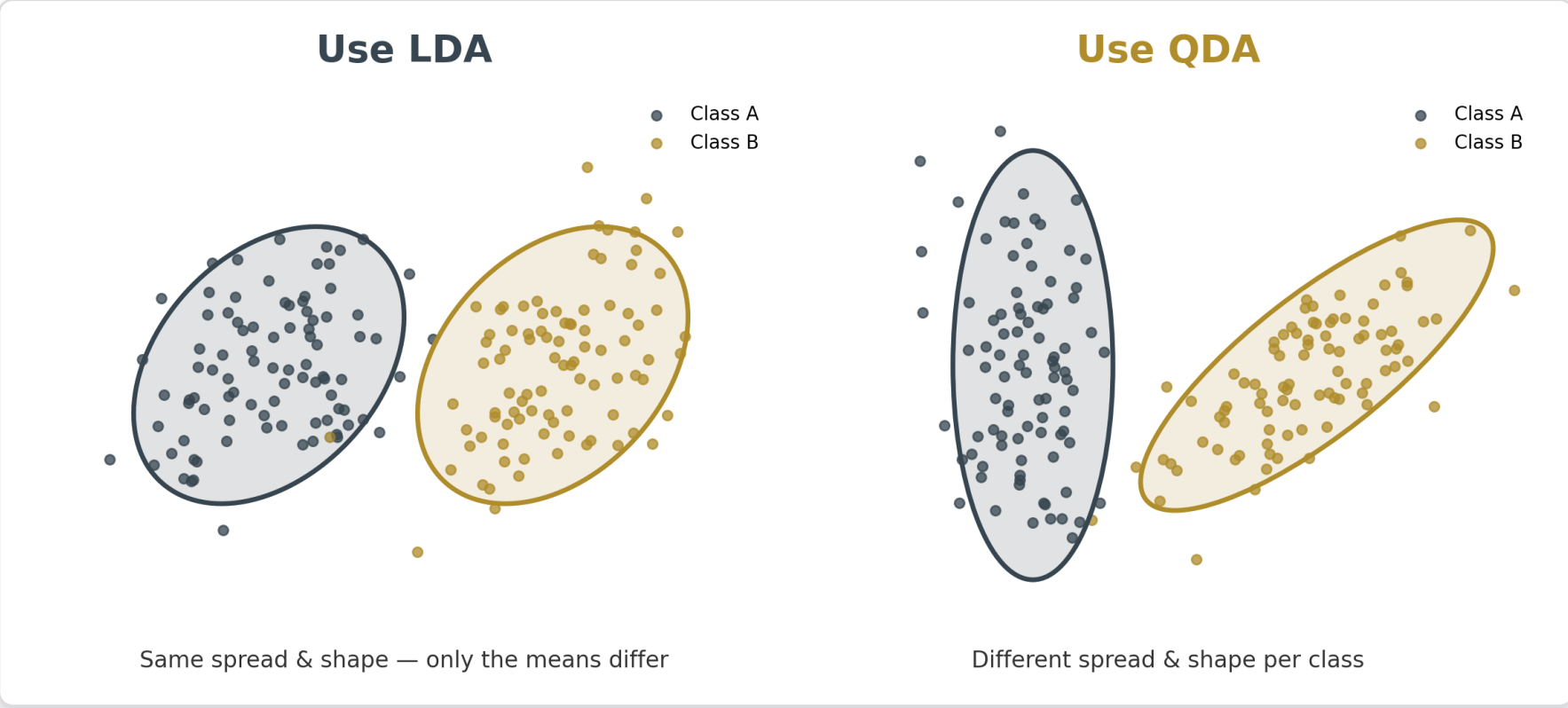
Δ

LDA

Useful when n is small, classes are well-separated, and Gaussian assumptions are reasonable.
Also preferred when K > 2.

When should you use LDA vs. QDA?

Look at the shape of each class's point cloud, not just where it's centered:

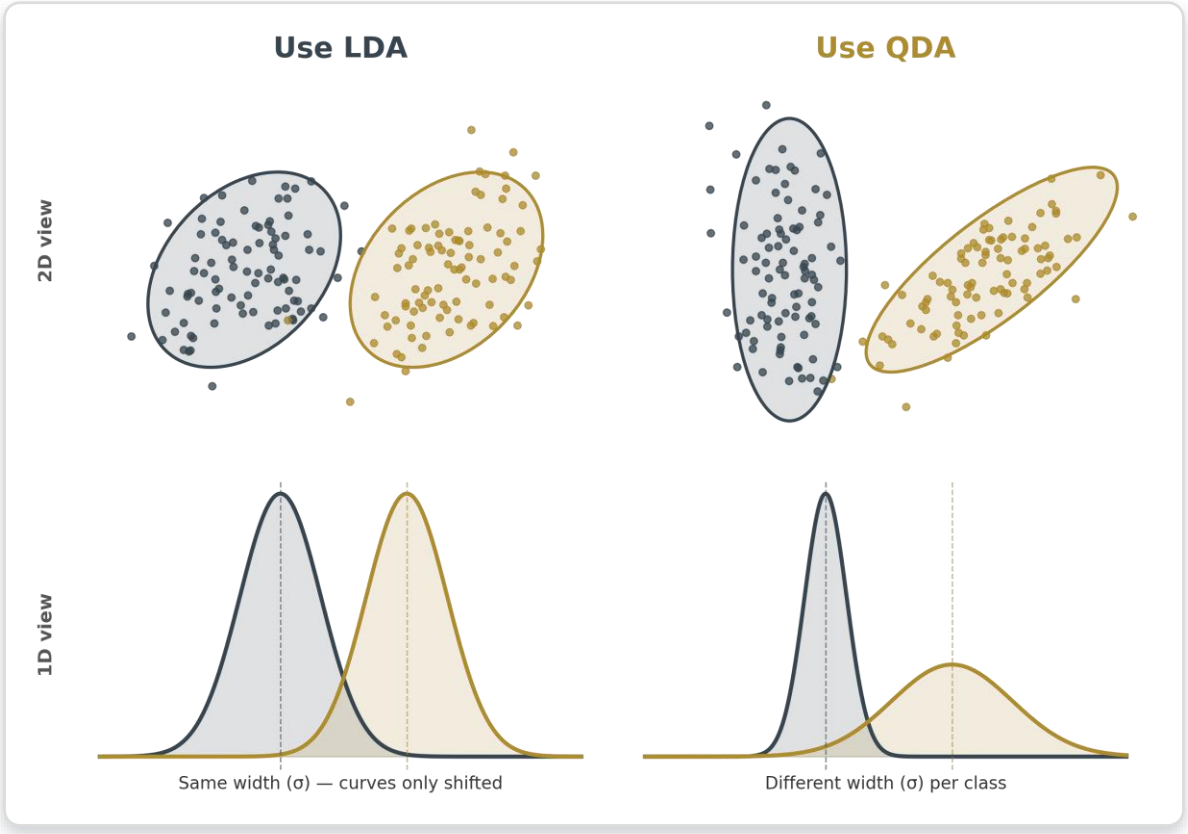


Left: the two ellipses are the same shape and orientation, just shifted, shared matrix would distort both

Right: the ellipses have different shapes and orientations, forcing one one shared covariance matrix describes both classes well. classes.

When should you use LDA vs. QDA?

Look at the shape of each class's point cloud, not just where it's centered:



Left: same covariance for both classes: in 2D, identical ellipses; in 1D, identical bell curves, just shifted.

Right: different covariance per class: the ellipses and the bell curves both change shape, not just position.

FURTHER RESOURCES

Where to learn more

A few video walkthroughs that cover the same material from a different angle:

ISLP

- Statistical Learning: 4.2 Logistic Regression
- Statistical Learning: 4.5 Discriminant Analysis

StatQuest

- StatQuest — Logistic Regression, Clearly Explained
- StatQuest: Linear Discriminant Analysis (LDA) clearly explained.

KEY TAKEAWAYS

Wrapping up

- Classification predicts a qualitative Y; plain linear regression doesn't respect the [0,1] probability range
- Logistic Regression models p(x) with the sigmoid, fit by Maximum Likelihood (always valid probabilities)
- Multiple predictors can reveal confounding a single-variable model hides (student & balance)
- LDA models each class as Gaussian and applies Bayes' theorem. QDA relaxes the equal-covariance assumption
- Accuracy alone can mislead on imbalanced classes: check the confusion matrix, and consider the decision threshold